{coding academy

Place Keeper

ES6 & HTML5

General

Our app consists of three HTML pages:

- **index.html** – the App's home page with navigation links to the two other pages.
- **user-prefs.html** – displays a `<form>` for collecting user preferences. These preferences determine how various parts of the app are displayed.
- **map.html** – displays a list of places saved by the user and a map.

Guidelines

Remember to use the <section, nav, main, aside, header, footer> semantic elements

Use the MVC pattern to shape your app, you should have the following services:

- `utilService` – *general utility functions.*
- `userService` – *manages saving and reading the user's preferences*
- `placeService` – *manages the place entity CRUDL*

Part 1 – Modern HTML

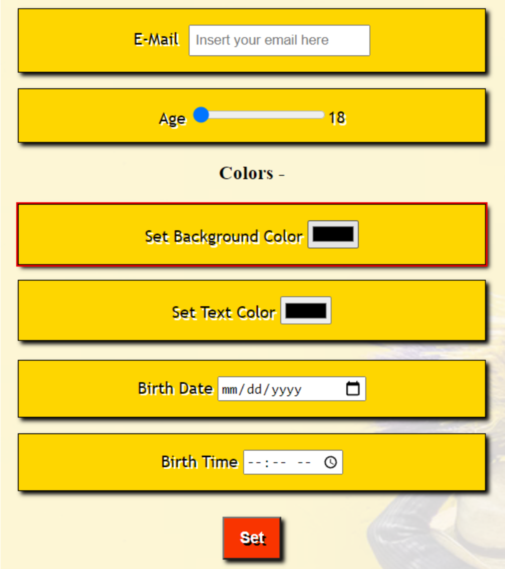
index.html

This is a simple home page with some graphics and a welcome message, something like: *Find your way back to your best places*

Use a Video element to show a nice HERO in the page

Add navigation links to the other (two) pages: user-settings.html and places.html

user-settings.html



Here we will use a <form> to get the user settings and save them to localStorage using the userService.

The **user** object will finally look like that

```
const user = {
    email : '',
    txtColor : '',
    bgColor : '',
    Age : '',
    birthDate: '',
    birthTime: ''
}
```

* Master tip: it's best to **start simple** with the first two properties – `email` and `txtColor`

The application should use the colors provided by the user and show the homepage (index.html) accordingly. (inside the onInit of the home-page, use the data saved in the storage to show the user preferences)
TIP: use: *userService.load()*

Development Flow

Step 1 - Colors

Use HTML5 color <input> to let the user set its background and text color of the pages.
TIP: use: *userService.save(userData)*

Step 2 – Date and Time

Use HTML5 *date* and *time* <input>s to let the user set his exact birth time, In the homepage render the user's birthtime

Step 3 – Wrap in a form

Put those inputs in a <form>, and on submit, use a service to keep them in a localstorage object: userData

TIP: you will need event.preventDefault in the onsubmit event handler.

Step 4 – Add some more inputs

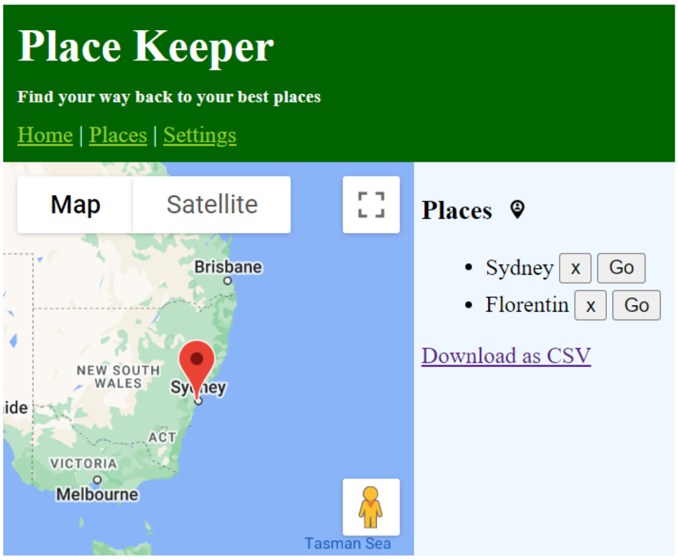
1. Add a *required* email <input>
2. Add a *range* <input> to let the user select his age: 18->120

places.html

Here we will show a **map** for now.
Later we will allow the user to manage his places.

Tips:

- Use the 'create Google Api' doc to create an API key and secure it.
- This exercise involves self learning and reading documentation for a new API



Show a map

1. Generate your Google Maps API key
   (see directions at the following link : https://scribehow.com/shared/Generate_a_Google_Maps_API_Key__rweWAZV_TTasKfGLbs15dw)
2. Show a map centered at **Eilat**
   - Copy the code needed for showing a simple map
   - you can use an online tool (such as this) for getting the lat-lng for Eilat

Bonuses

1. In the user-settings.html
   - add another input: gender, that is based on a datalist with the options: Male, Female, Other
   - Add validations to make sure the user filled out all of the inputs correctly before submitting the form.
   - Add custom validation: validate the provided user age matches the provided birth year
2. **Connect the user-settings form to the map**
   In the user-settings.html add 2 more fields to the form and to the user object.
   - InitialZoom (number)
   - InitialLocation (an object containing lat , lng - text)

   In the map.html – if the user has selected an initialZoom or initialLocation – set up the map by the user preferences.

3. Create more pages and try out some HTML5 features we have covered

Part 2 – Modern JS

Use ES6 throughout your code: destructering, arrow functions, default parameter values, let, const, etc.
Refactor the code you already have written to ES6.

Implement full working CRUDL to allow the user to manage his favorite places.

Development Flow

Step 1 – places list and remove

Show the list and allow the user to remove a place.
Use a placeService that manages the place entity, a place object looks like that:

```
{id: '1p2', lat: 32.1416, lng: 34.831213, name: 'Pukis house'}
```

- Start from rendering 2 places on the page
- Setup your place.controller
  - `function onInit() {}`
  - `function renderPlaces() {}`
  - `function onRemovePlace(placeId) {}`
- Setup the place.service
  - `function getPlaces() {}`
  - `function removePlace(placeId) {}`
  - `function addPlace(name, lat, lng, zoom) {}`
  - `function getPlaceById(placeId) {}`
  - `function _createPlace(name, lat, lng, zoom) {}`
  - `function _createPlaces() {}`
- Render the list and check that your functions work

Step 2 – Pan to a place

When a user clicks a button to go to a place, the map is moved and zoomed on the selected place

```
function onPanToPlace(placeId) {
    const place = getPlaceById(placeId)
    gMap.setCenter({ lat: place.lat, lng: place.lng})
    gMap.setZoom(place.zoom)
}
```

Step 3 – Create a place

When a user clicks on the map, the user is prompted to enter a name and a new place is saved to storage
here is some code to put you in the right direction:

```
gMap.addListener('click', ev => {
        const name = prompt('Place name?', 'Place 1')
        const lat = ev.latLng.lat()
        const lng = ev.latLng.lng()
        addPlace(name, lat, lng, gMap.getZoom())
        renderPlaces()
})
```

Step 4 – User location

Add a button to the map (Using google map documentation on how to do that).
when user clicks the button, get his current location and center the map accordingly.
Tip: Use the code shown in class (Modern HTML).

Step 5 – Markers

When the map is ready, and also when places are added / removed, we call the renderMarkers function:

```
function renderMarkers() {
    const places = getPlaces()
    // remove previous markers
    gMarkers.forEach(marker => marker.setMap(null))
    // every place is creating a marker
    gMarkers = places.map(place => {
        return new google.maps.marker.AdvancedMarkerElement({
            position: place,
            map: gMap,
            title: place.name
        })
    })
}
```

Step 6 - Finalize the app

1. Add navigation links to all pages.
2. Let the user download a CSV of the places
3. Make the app look great for both desktop and mobile views.

Bonus

Replace the prompt for new place name with a nice modal (Using dialog element)